

Deep Learning Notes

ANN, CNN, RNN and GAN — Theory, Parameters and Code

July 23, 2026

Contents

1	Introduction to Deep Learning	2
2	Artificial Neural Network (ANN)	2
2.1	Theory	2
2.2	Code Example (Keras/TensorFlow)	3
3	Convolutional Neural Network (CNN)	4
3.1	Theory	4
3.2	Code Example (Keras/TensorFlow)	4
4	Recurrent Neural Network (RNN)	6
4.1	Theory	6
4.2	Code Example (Keras/TensorFlow — LSTM)	6
5	Generative Adversarial Network (GAN)	8
5.1	Theory	8
5.2	Code Example (Keras/TensorFlow — Simple GAN)	8
6	Comparison Summary	10
7	Cross Validation	11
7.1	Definition	11
7.2	k-Fold Cross Validation	11
7.3	Formula	11
7.4	Advantages	11
7.5	Disadvantages	12
7.6	Python Example	12
7.7	Common Cross Validation Techniques	12
8	Hyperparameter Tuning	12
8.1	Definition	12
8.2	Parameters vs Hyperparameters	13
8.3	Common Hyperparameters	13
8.4	Why Hyperparameter Tuning?	13
9	GridSearchCV	13
9.1	Definition	13
9.2	Example	13
9.3	Python Example	14

10 RandomizedSearchCV	14
10.1 Definition	14
10.2 Python Example	14
11 GridSearchCV vs RandomizedSearchCV	15
12 Decision Tree Hyperparameter Tuning	15
12.1 Important Parameters	15
12.2 Python Example	15
13 Interview Questions	15

1 Introduction to Deep Learning

Deep Learning is a subfield of Machine Learning that uses multi-layered artificial neural networks to automatically learn hierarchical representations of data. Instead of hand-crafting features, deep networks learn features directly from raw data (images, text, audio, etc.) through successive layers of transformation.

Why “Deep”?

A network is called *deep* when it has more than one hidden layer between the input and output. Each layer transforms its input using a learned weight matrix W , a bias vector b , and a non-linear activation function σ :

$$a^{(l)} = \sigma(W^{(l)}a^{(l-1)} + b^{(l)})$$

Stacking many such layers allows the network to approximate highly non-linear functions (Universal Approximation Theorem).

2 Artificial Neural Network (ANN)

2.1 Theory

An ANN (also called a Multi-Layer Perceptron, MLP) consists of an input layer, one or more hidden layers, and an output layer. Each neuron computes a weighted sum of inputs, adds a bias, and applies an activation function.

Forward propagation for a single neuron:

$$z = \sum_{i=1}^n w_i x_i + b, \quad a = \sigma(z)$$

Common activation functions:

$$\text{Sigmoid: } \sigma(z) = \frac{1}{1 + e^{-z}}, \quad \text{ReLU: } \sigma(z) = \max(0, z), \quad \text{Tanh: } \sigma(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Loss function (example — Mean Squared Error):

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Backpropagation uses the chain rule to compute $\frac{\partial L}{\partial W^{(l)}}$ for every layer, and weights are updated using gradient descent:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

where η is the learning rate.

Key Parameters / Hyperparameters

- **Learning rate (η):** step size for weight updates (e.g. 0.001–0.1)
- **Number of hidden layers & neurons:** controls model capacity
- **Activation function:** ReLU, Sigmoid, Tanh, etc.

- **Batch size:** number of samples per gradient update (e.g. 32, 64, 128)
- **Epochs:** number of full passes over the training data
- **Optimizer:** SGD, Adam, RMSprop
- **Weight initialization:** Xavier/Glorot, He initialization
- **Regularization:** L1/L2 penalty, Dropout rate

2.2 Code Example (Keras/TensorFlow)

```
1 import tensorflow as tf
2 from tensorflow import keras
3 from tensorflow.keras import layers
4
5 # Build a simple ANN
6 model = keras.Sequential([
7     layers.Dense(64, activation='relu', input_shape=(20,)),
8     layers.Dense(32, activation='relu'),
9     layers.Dense(1, activation='sigmoid') # binary classification
10 ])
11
12 model.compile(optimizer='adam',
13               loss='binary_crossentropy',
14               metrics=['accuracy'])
15
16 model.summary()
17
18 history = model.fit(X_train, y_train,
19                    epochs=50,
20                    batch_size=32,
21                    validation_split=0.2)
```

3 Convolutional Neural Network (CNN)

3.1 Theory

CNNs are designed for grid-like data such as images. They use *convolution* operations to extract spatial features (edges, textures, shapes) instead of fully connecting every pixel to every neuron.

Convolution operation:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n)$$

where I is the input image and K is the kernel/filter.

Output size formula:

$$O = \left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1$$

where W = input size, F = filter size, P = padding, S = stride.

A typical CNN architecture stacks:

Conv $\rightarrow$ Activation (ReLU) $\rightarrow$ Pooling $\rightarrow \dots \rightarrow$ Flatten $\rightarrow$ Fully Connected $\rightarrow$ Output

Pooling (Max Pooling) reduces spatial dimensions and adds translation invariance:

$$\text{MaxPool}(x) = \max_{(i,j) \in \text{window}} x_{i,j}$$

Key Parameters / Hyperparameters

- **Filter/Kernel size:** e.g. 3×3 , 5×5
- **Number of filters:** determines number of feature maps (e.g. 32, 64, 128)
- **Stride:** step size of the filter (e.g. 1, 2)
- **Padding:** same (keeps size) or valid (no padding)
- **Pooling size:** e.g. 2×2 max pooling
- **Number of Conv layers:** depth of feature extraction
- **Dropout rate:** to reduce overfitting (e.g. 0.25–0.5)
- **Batch normalization:** stabilizes and speeds up training

3.2 Code Example (Keras/TensorFlow)

```

1 from tensorflow.keras import layers, models
2
3 model = models.Sequential([
4     layers.Conv2D(32, (3,3), activation='relu', input_shape=(64,64,3)),
5     layers.MaxPooling2D((2,2)),
6
7     layers.Conv2D(64, (3,3), activation='relu'),
8     layers.MaxPooling2D((2,2)),
9
10    layers.Conv2D(128, (3,3), activation='relu'),
11    layers.MaxPooling2D((2,2)),
12
13    layers.Flatten(),

```

```
14     layers.Dense(128, activation='relu'),
15     layers.Dropout(0.5),
16     layers.Dense(10, activation='softmax')    # 10-class classification
17 ])
18
19 model.compile(optimizer='adam',
20               loss='sparse_categorical_crossentropy',
21               metrics=['accuracy'])
22
23 model.fit(X_train, y_train, epochs=20, batch_size=64,
24         validation_data=(X_val, y_val))
```

4 Recurrent Neural Network (RNN)

4.1 Theory

RNNs are designed for sequential data (text, time series, speech). Unlike ANNs/CNNs, an RNN has a *hidden state* that carries information from previous time steps, giving it a form of memory.

Recurrence relation:

$$h_t = \sigma(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \quad y_t = W_{hy}h_t + b_y$$

where h_t is the hidden state at time t and x_t is the input at time t .

Problem — Vanishing/Exploding Gradients: During backpropagation through time (BPTT), gradients are multiplied repeatedly by the recurrent weight matrix, which can cause them to vanish or explode over long sequences.

LSTM (Long Short-Term Memory) solves this using gates:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (\text{forget gate})$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (\text{input gate})$$

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (\text{cell state update})$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(C_t)$$

Key Parameters / Hyperparameters

- **Sequence length / timesteps:** how many past steps are fed in
- **Hidden units:** size of hidden state vector (e.g. 64, 128, 256)
- **Number of recurrent layers:** stacked LSTM/GRU layers
- **Cell type:** Vanilla RNN, LSTM, GRU
- **Return sequences:** whether to return output at every timestep
- **Dropout / Recurrent dropout:** regularization inside recurrent layers
- **Bidirectionality:** process sequence forward and backward
- **Gradient clipping:** limits gradient norm to avoid explosion

4.2 Code Example (Keras/TensorFlow — LSTM)

```

1 from tensorflow.keras import layers, models
2
3 model = models.Sequential([
4     layers.Embedding(input_dim=10000, output_dim=128, input_length=100),
5     layers.LSTM(128, return_sequences=True, dropout=0.2, recurrent_dropout=0.2)
6     ,
7     layers.LSTM(64, dropout=0.2),
8     layers.Dense(1, activation='sigmoid') # e.g. sentiment classification
9 ])
10
11 model.compile(optimizer='adam',
12               loss='binary_crossentropy',

```

```
12         metrics=['accuracy'])
13
14 model.fit(X_train, y_train, epochs=10, batch_size=32,
15         validation_data=(X_val, y_val))
```

5 Generative Adversarial Network (GAN)

5.1 Theory

A GAN consists of two networks trained together in a zero-sum (adversarial) game:

- **Generator (G):** takes random noise z and generates fake data $G(z)$ that tries to look real.
- **Discriminator (D):** takes real or fake data and outputs a probability that the input is real.

Minimax objective:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

Training process:

1. Sample real data x and noise z ; generate fake data $G(z)$.
2. Train D to maximize $\log D(x) + \log(1 - D(G(z)))$ (distinguish real vs. fake).
3. Train G to minimize $\log(1 - D(G(z)))$, i.e. fool the discriminator.
4. Repeat until $D(G(z)) \approx 0.5$ (Nash equilibrium — generator produces realistic data).

Key Parameters / Hyperparameters

- **Latent (noise) vector size:** dimensionality of z (e.g. 100)
- **Generator/Discriminator architecture:** layers, filters (often CNN-based — DC-GAN)
- **Learning rate (separate for G and D):** e.g. 2×10^{-4}
- **Batch size:** e.g. 64, 128
- **Optimizer:** typically Adam with $\beta_1 = 0.5$
- **Loss function:** Binary cross-entropy (vanilla GAN), Wasserstein loss (WGAN)
- **Number of training epochs / steps:** GANs often need many epochs
- **Label smoothing:** helps stabilize discriminator training

5.2 Code Example (Keras/TensorFlow — Simple GAN)

```

1 import tensorflow as tf
2 from tensorflow.keras import layers, models
3
4 latent_dim = 100
5
6 # ---- Generator ----
7 generator = models.Sequential([
8     layers.Dense(128, activation='relu', input_dim=latent_dim),
9     layers.Dense(256, activation='relu'),
10    layers.Dense(784, activation='tanh'),    # e.g. 28x28 flattened image
11    layers.Reshape((28, 28, 1))
12 ])

```

```
13
14 # ---- Discriminator ----
15 discriminator = models.Sequential([
16     layers.Flatten(input_shape=(28, 28, 1)),
17     layers.Dense(256, activation='relu'),
18     layers.Dense(128, activation='relu'),
19     layers.Dense(1, activation='sigmoid')
20 ])
21 discriminator.compile(optimizer='adam', loss='binary_crossentropy',
22                       metrics=['accuracy'])
23
24 # ---- Combined GAN (Generator + frozen Discriminator) ----
25 discriminator.trainable = False
26 gan_input = layers.Input(shape=(latent_dim,))
27 fake_image = generator(gan_input)
28 gan_output = discriminator(fake_image)
29 gan = models.Model(gan_input, gan_output)
30 gan.compile(optimizer='adam', loss='binary_crossentropy')
31
32 # ---- Training loop (simplified) ----
33 for epoch in range(epochs):
34     noise = tf.random.normal([batch_size, latent_dim])
35     fake_images = generator.predict(noise)
36     real_images = get_real_batch(batch_size) # your data loader
37
38     d_loss_real = discriminator.train_on_batch(real_images, tf.ones((batch_size, 1)))
39     d_loss_fake = discriminator.train_on_batch(fake_images, tf.zeros((batch_size, 1)))
40
41     noise = tf.random.normal([batch_size, latent_dim])
42     g_loss = gan.train_on_batch(noise, tf.ones((batch_size, 1)))
```

6 Comparison Summary

Model	Best suited for	Key idea	Typical use case
ANN	Structured/tabular data	Fully connected layers	Classification, regression
CNN	Images, grid data	Convolution + pooling	Image recognition, object detection
RNN/LSTM	Sequential data	Recurrent hidden state, gates	Text, time series, speech
GAN	Data generation	Adversarial G vs. D training	Image synthesis, data augmentation

Table 1: Quick comparison of core deep learning architectures

7 Cross Validation

7.1 Definition

Cross Validation is a model evaluation technique used to estimate the performance of a machine learning model on unseen data. Instead of splitting the dataset only once into training and testing sets, the data is divided into multiple parts called **folds**. Each fold is used once as the testing set while the remaining folds are used for training.

Purpose:

- Evaluate model performance more reliably.
- Reduce overfitting.
- Utilize the entire dataset efficiently.
- Compare different machine learning models.

7.2 k-Fold Cross Validation

In k-Fold Cross Validation:

1. Divide the dataset into k equal folds.
2. Select one fold as the testing set.
3. Train the model on the remaining $k - 1$ folds.
4. Repeat the process until each fold has been used as the testing set.
5. Compute the average accuracy.

Example for $k = 5$

```
Fold 1 -> Test
Fold 2 -> Train
Fold 3 -> Train
Fold 4 -> Train
Fold 5 -> Train
```

Repeat until every fold becomes the test fold.

7.3 Formula

Average Accuracy

$$Accuracy = \frac{Accuracy_1 + Accuracy_2 + \dots + Accuracy_k}{k}$$

7.4 Advantages

- Better estimate of model performance.
- Reduces overfitting.
- Efficient use of data.
- Less biased evaluation.

7.5 Disadvantages

- Computationally expensive.
- Takes more training time.

7.6 Python Example

```

1 from sklearn.datasets import load_iris
2 from sklearn.tree import DecisionTreeClassifier
3 from sklearn.model_selection import cross_val_score
4
5 iris = load_iris()
6
7 X = iris.data
8 y = iris.target
9
10 model = DecisionTreeClassifier(random_state=42)
11
12 scores = cross_val_score(
13     estimator=model,
14     X=X,
15     y=y,
16     cv=5,
17     scoring='accuracy'
18 )
19
20 print("Accuracy of each fold:")
21 print(scores)
22
23 print("Average Accuracy:")
24 print(scores.mean())

```

Expected Output

Accuracy of each fold:
[0.96 1.00 0.93 0.96 1.00]

Average Accuracy:
0.97

7.7 Common Cross Validation Techniques

Technique	Description
Train-Test Split	Single train-test division
K-Fold CV	Dataset divided into K equal folds
Stratified K-Fold	Maintains class distribution
Leave-One-Out CV	One sample used for testing each time
Repeated K-Fold	K-Fold repeated multiple times

8 Hyperparameter Tuning

8.1 Definition

Hyperparameters are the settings of a machine learning algorithm that are specified before training begins. They are not learned automatically by the model.

Hyperparameter Tuning is the process of finding the best combination of hyperparameter values to maximize model performance.

8.2 Parameters vs Hyperparameters

Parameters	Hyperparameters
Learned during training	Set before training
Examples: Weights, Biases	Examples: max_depth, learning_rate
Automatically updated	Manually selected or searched

8.3 Common Hyperparameters

Algorithm	Hyperparameters
Decision Tree	max_depth, criterion
Random Forest	n_estimators, max_depth
KNN	n_neighbors
SVM	C, gamma
Neural Network	epochs, batch_size, learning_rate
Logistic Regression	C

8.4 Why Hyperparameter Tuning?

- Improve model accuracy.
- Reduce overfitting.
- Reduce underfitting.
- Find the optimal model.

9 GridSearchCV

9.1 Definition

GridSearchCV exhaustively searches through every possible combination of the specified hyperparameter values using Cross Validation.

9.2 Example

Suppose

```
criterion = [gini, entropy]
```

```
max_depth = [2,3,4]
```

Possible combinations

```
(gini,2)
(gini,3)
(gini,4)
(entropy,2)
(entropy,3)
(entropy,4)
```

GridSearchCV evaluates all six combinations.

9.3 Python Example

```
1 from sklearn.datasets import load_iris
2 from sklearn.tree import DecisionTreeClassifier
3 from sklearn.model_selection import GridSearchCV
4
5 iris = load_iris()
6
7 X = iris.data
8 y = iris.target
9
10 parameters = {
11     "criterion":["gini","entropy"],
12     "max_depth":[2,3,4,5]
13 }
14
15 grid = GridSearchCV(
16     estimator=DecisionTreeClassifier(),
17     param_grid=parameters,
18     cv=5,
19     scoring="accuracy"
20 )
21
22 grid.fit(X,y)
23
24 print("Best Parameters:")
25 print(grid.best_params_)
26
27 print("Best Accuracy:")
28 print(grid.best_score_)
```

10 RandomizedSearchCV

10.1 Definition

RandomizedSearchCV randomly samples a fixed number of parameter combinations rather than checking every possible combination.

It is faster than GridSearchCV, especially when the parameter search space is large.

10.2 Python Example

```
1 from sklearn.datasets import load_iris
2 from sklearn.tree import DecisionTreeClassifier
3 from sklearn.model_selection import RandomizedSearchCV
4
5 iris = load_iris()
6
7 X = iris.data
8 y = iris.target
9
10 parameters = {
11     "criterion":["gini","entropy"],
12     "max_depth":[2,3,4,5,6],
13     "min_samples_split":[2,4,6,8]
14 }
15
16 random = RandomizedSearchCV(
17     estimator=DecisionTreeClassifier(),
18     param_distributions=parameters,
```

```

19     cv=5,
20     n_iter=5,
21     random_state=42
22 )
23
24 random.fit(X,y)
25
26 print(random.best_params_)
27 print(random.best_score_)

```

11 GridSearchCV vs RandomizedSearchCV

Feature	GridSearchCV	RandomizedSearchCV
Search Type	Exhaustive	Random
Speed	Slow	Fast
Accuracy	Higher	Good
Large Search Space	No	Yes
Computational Cost	High	Low

12 Decision Tree Hyperparameter Tuning

12.1 Important Parameters

Parameter	Description
criterion	Splitting criterion (gini or entropy)
max_depth	Maximum depth of tree
min_samples_split	Minimum samples required to split a node
min_samples_leaf	Minimum samples required in a leaf node
max_features	Number of features considered for splitting
splitter	best or random
ccp_alpha	Cost complexity pruning parameter

12.2 Python Example

```

1 from sklearn.tree import DecisionTreeClassifier
2
3 model = DecisionTreeClassifier(
4     criterion="entropy",
5     max_depth=4,
6     min_samples_split=5,
7     min_samples_leaf=2,
8     splitter="best",
9     random_state=42
10 )
11
12 model.fit(X,y)

```

13 Interview Questions

1. What is Cross Validation?
2. Why is k-Fold Cross Validation used?

3. What is Stratified K-Fold?
4. What are Hyperparameters?
5. Difference between Parameters and Hyperparameters?
6. What is Hyperparameter Tuning?
7. Explain GridSearchCV.
8. Explain RandomizedSearchCV.
9. Which is faster: GridSearchCV or RandomizedSearchCV?
10. Why do we use Cross Validation with GridSearchCV?